

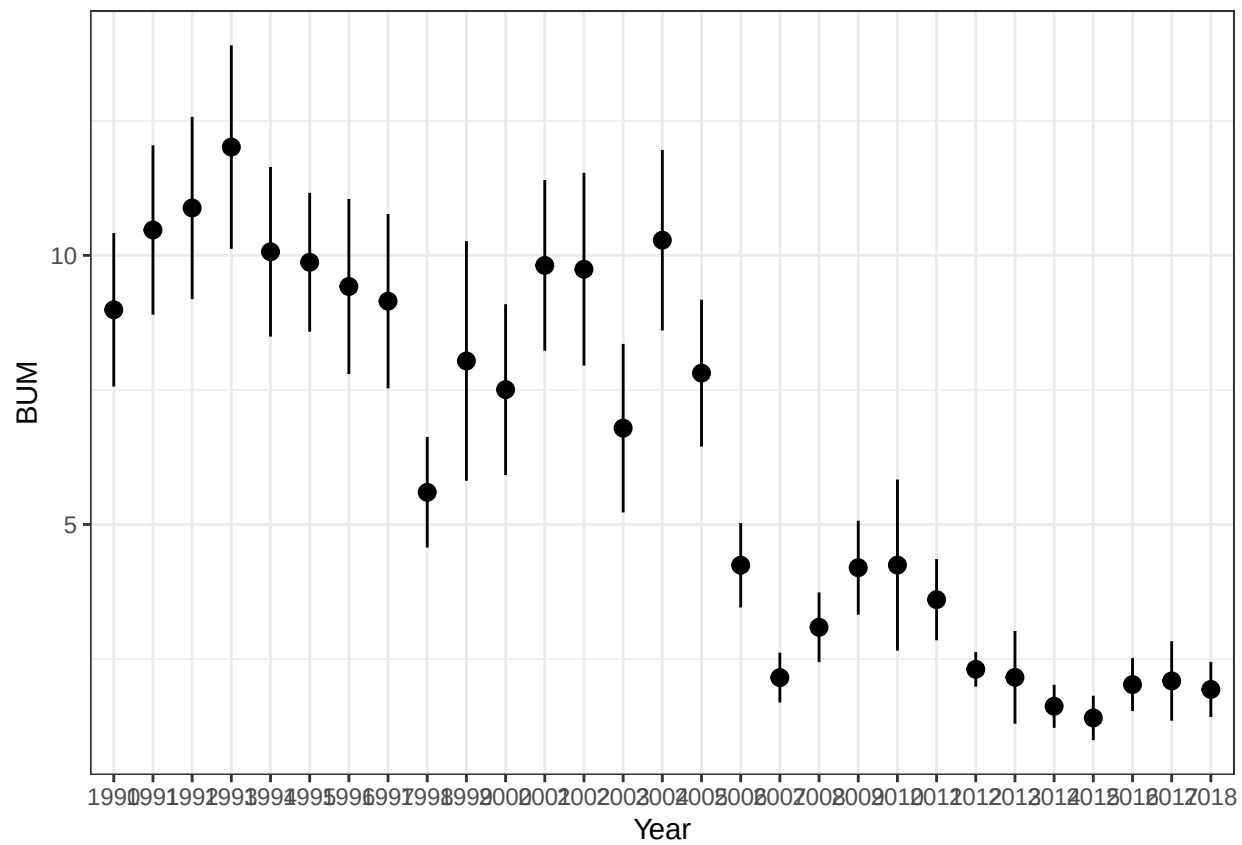
Catch, effort and CPUE

Beth Babcock

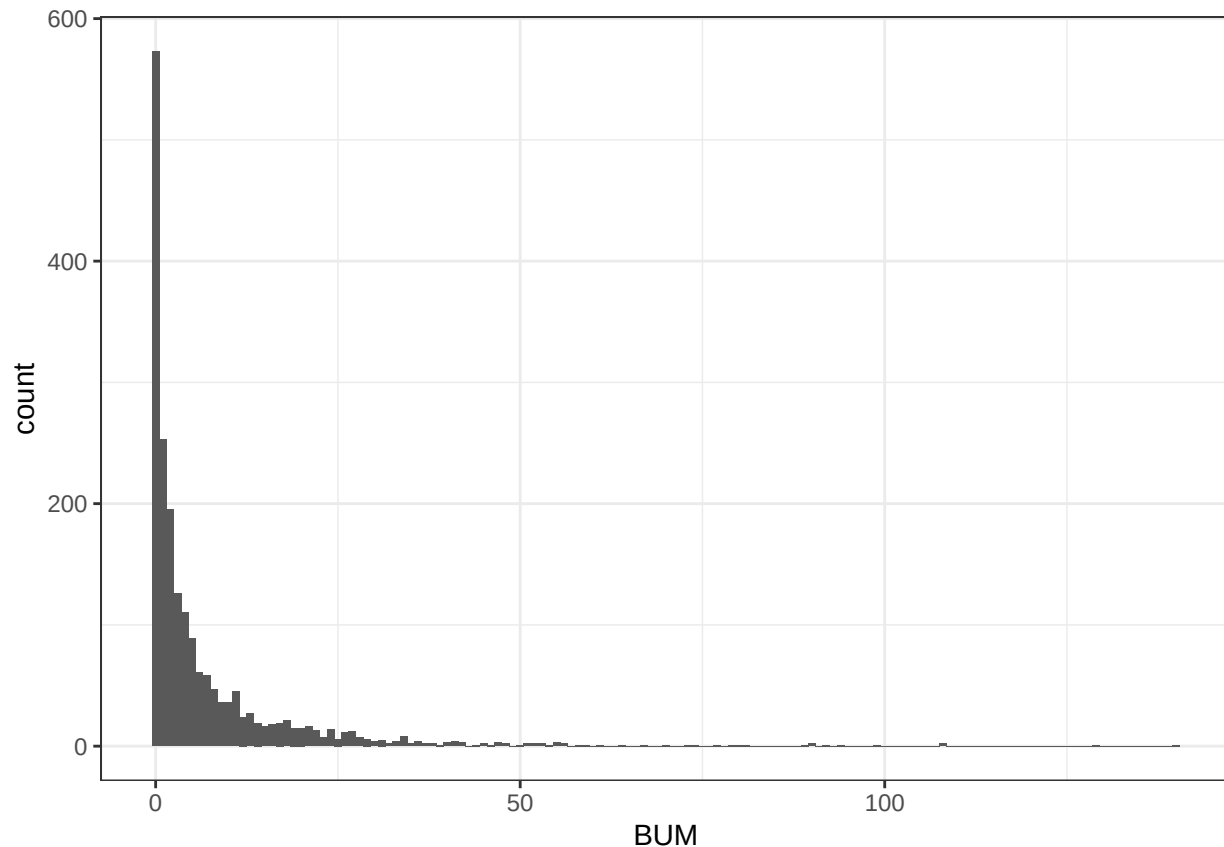
2025-11-04

```
library(tidyverse)
library(glmmTMB)
library(MuMIn)
library(DHARMA)
library(mgcv)
theme_set(theme_bw())
```

```
obsdat<-read.csv("obsdat.csv") %>%
  mutate(Year=factor(Year),
         fleet=factor(fleet),
         season=factor(season),
         area=factor(area))
ggplot(obsdat,
       aes(x=Year,y=BUM))+
  stat_summary(fun.data="mean_se")
```



```
ggplot(obsdat,aes(x=BUM))+
  geom_histogram(binwidth=1)
```



Poisson vs. nb 1 or 2

```
df<-expand.grid(draw=1:1000,model=c("Poisson","Negative Binomial")) %>%
  rowwise()%>%
  mutate(Draw=ifelse(model=="Poisson",rpois(1,10),rnbinom(1,mu=10,size=0.5)))
summary(df)
```

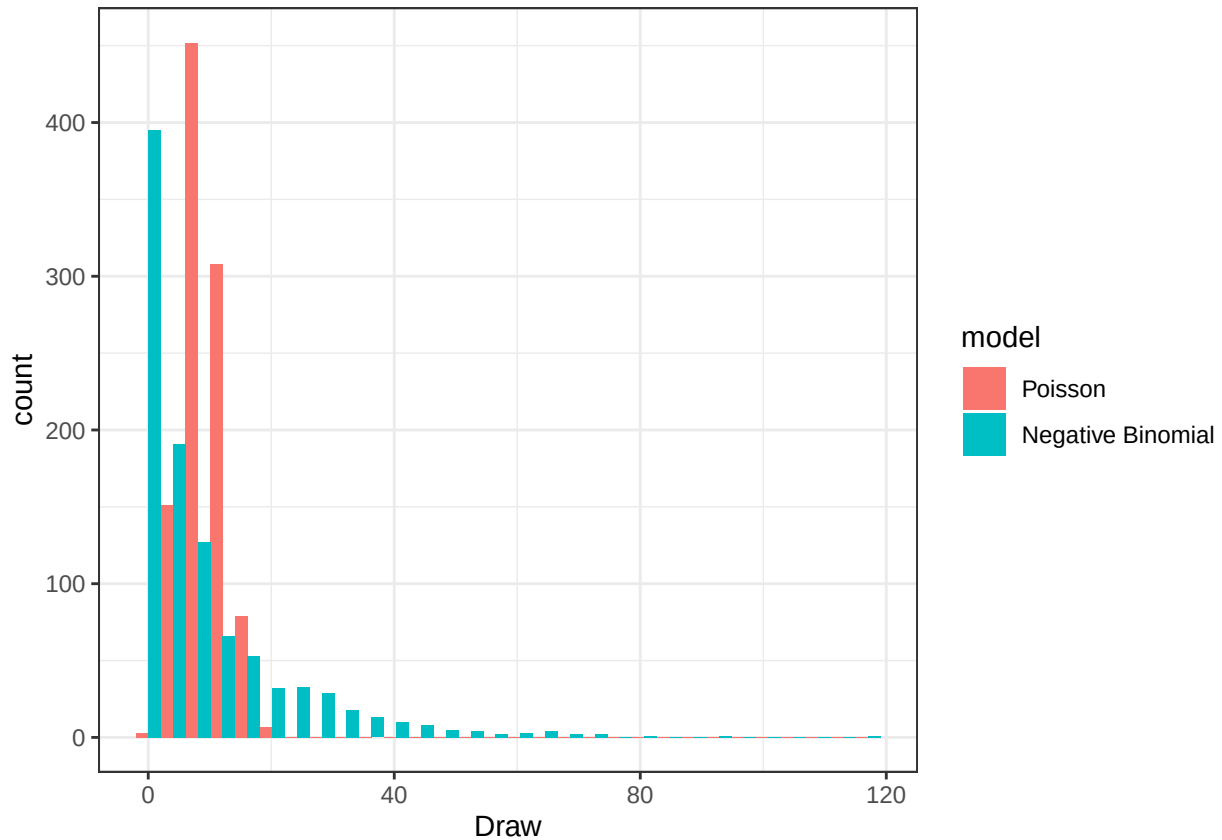
##	draw	model	Draw
##	Min. : 1.0	Poisson :1000	Min. : 0.000
##	1st Qu.: 250.8	Negative Binomial:1000	1st Qu.: 4.000
##	Median : 500.5		Median : 8.000
##	Mean : 500.5		Mean : 9.669
##	3rd Qu.: 750.2		3rd Qu.: 12.000
##	Max. : 1000.0		Max. : 117.000

```
df%>% group_by(model)%>%
  summarize(mean=mean(Draw),
            var=var(Draw))
```

```
## # A tibble: 2 x 3
##   model      mean  var
##   <fct>    <dbl> <dbl>
## 1 Poisson      9.82  10.3
## 2 Negative Binomial 9.52 177.
```

```
ggplot(df,aes(x=Draw,fill=model))+
  geom_histogram(position="dodge")
```

'stat_bin()' using 'bins = 30'. Pick better value 'binwidth'.



Run a poisson model and look at the residuals

```
poissonFit<-glmmTMB(BUM~Year+area+season+offset(log(sets)),
  data=obsdat,
  family="poisson")
summary(poissonFit)
```

```
## Family: poisson ( log )
## Formula:      BUM ~ Year + area + season + offset(log(sets))
## Data: obsdat
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  19625.0  19809.5  -9779.5   19559.0     1950
##
##
## Conditional model:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.34489    0.03712  -9.290 < 2e-16 ***
## Year1991     0.16583    0.04831   3.433 0.000598 ***
```

```

## Year1992      0.13237      0.05122      2.584 0.009754 **
## Year1993      0.14885      0.04785      3.111 0.001867 **
## Year1994      0.06250      0.04722      1.323 0.185702
## Year1995     -0.12230      0.04982     -2.455 0.014095 *
## Year1996     -0.23358      0.04808     -4.858 1.18e-06 ***
## Year1997     -0.32748      0.05248     -6.240 4.39e-10 ***
## Year1998     -0.56950      0.05882     -9.682 < 2e-16 ***
## Year1999     -0.25494      0.05340     -4.774 1.81e-06 ***
## Year2000     -0.35388      0.05391     -6.564 5.23e-11 ***
## Year2001     -0.32089      0.05475     -5.861 4.60e-09 ***
## Year2002     -0.29664      0.05403     -5.490 4.02e-08 ***
## Year2003     -0.58226      0.05656    -10.294 < 2e-16 ***
## Year2004     -0.28079      0.04905     -5.725 1.03e-08 ***
## Year2005     -0.61355      0.05746    -10.679 < 2e-16 ***
## Year2006     -1.01174      0.06673    -15.163 < 2e-16 ***
## Year2007     -1.70572      0.08841    -19.294 < 2e-16 ***
## Year2008     -1.24530      0.07366    -16.905 < 2e-16 ***
## Year2009     -0.96041      0.07188    -13.361 < 2e-16 ***
## Year2010     -0.99849      0.07343    -13.597 < 2e-16 ***
## Year2011     -1.23129      0.08075    -15.249 < 2e-16 ***
## Year2012     -1.54382      0.09573    -16.126 < 2e-16 ***
## Year2013     -1.49251      0.10870    -13.730 < 2e-16 ***
## Year2014     -1.95717      0.12239    -15.991 < 2e-16 ***
## Year2015     -2.05725      0.14314    -14.372 < 2e-16 ***
## Year2016     -1.69464      0.11816    -14.342 < 2e-16 ***
## Year2017     -1.62211      0.11124    -14.583 < 2e-16 ***
## Year2018     -1.70627      0.11178    -15.264 < 2e-16 ***
## areaS        -0.24011      0.01767    -13.588 < 2e-16 ***
## season2      -0.10627      0.02279     -4.662 3.13e-06 ***
## season3      -0.07117      0.02513     -2.832 0.004628 **
## season4      -0.35320      0.02341    -15.089 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

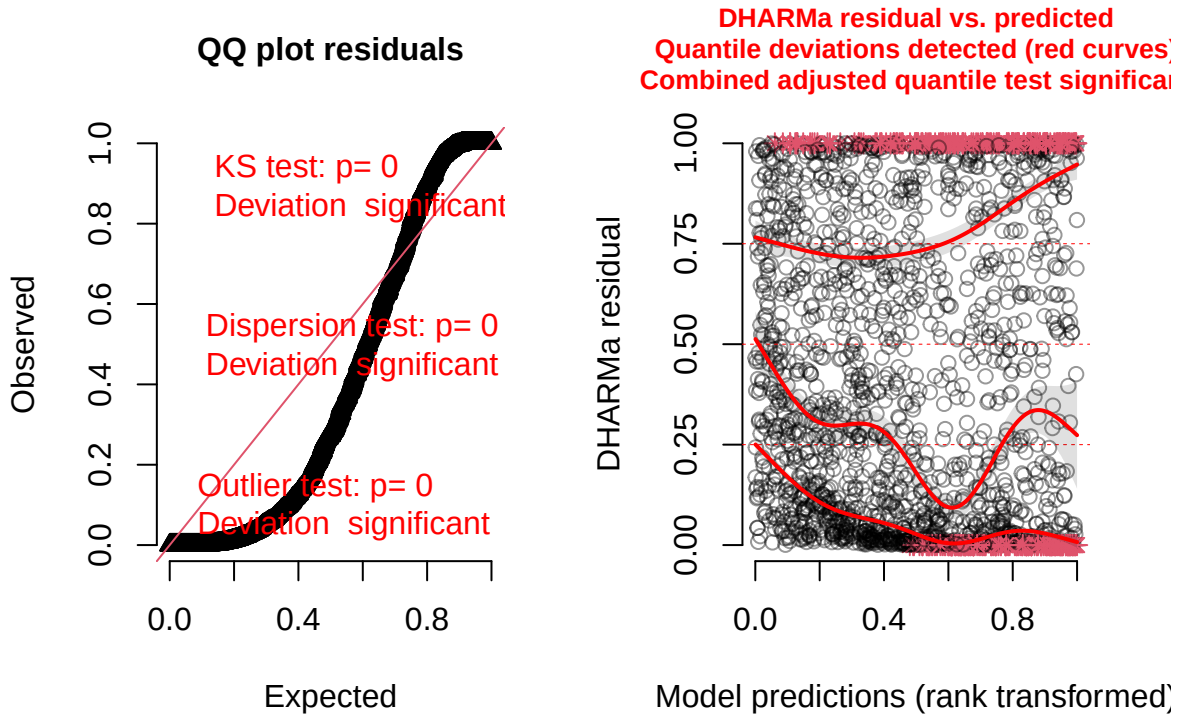
```

```
plot(simulateResiduals(poissonFit))
```

```
## DHARma:testOutliers with type = binomial may have inflated Type I error rates for integer-valued dis
```

```
## qu = 0.5, log(sigma) = -1.51611 : outer Newton did not converge fully.
```

DHARMa residual



Poisson doesn't work, try NB2

```
NB2fit<-glmmTMB(BUM~Year+area+season+offset(log(sets)),
                data=obsdat,
                family="nbinom2")
summary(NB2fit)
```

```
## Family: nbinom2 ( log )
## Formula:      BUM ~ Year + area + season + offset(log(sets))
## Data: obsdat
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  10272.5  10462.6   -5102.2   10204.5     1949
##
##
## Dispersion parameter for nbinom2 family (): 0.806
##
## Conditional model:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.06447    0.13897  -0.464  0.64269
## Year1991     0.29697    0.18060   1.644  0.10009
## Year1992     0.19664    0.19408   1.013  0.31097
## Year1993    -0.03041    0.18332  -0.166  0.86823
## Year1994    -0.03557    0.17505  -0.203  0.83897
## Year1995    -0.13192    0.18185  -0.725  0.46821
## Year1996    -0.41534    0.17449  -2.380  0.01730 *
```

```

## Year1997    -0.34489    0.19040   -1.811    0.07008  .
## Year1998    -0.80716    0.18937   -4.262    2.02e-05 ***
## Year1999    -0.37808    0.19106   -1.979    0.04783  *
## Year2000    -0.37781    0.18891   -2.000    0.04551  *
## Year2001    -0.29235    0.20591   -1.420    0.15567
## Year2002    -0.47145    0.20242   -2.329    0.01985  *
## Year2003    -0.83461    0.19281   -4.329    1.50e-05 ***
## Year2004    -0.50507    0.18295   -2.761    0.00577  **
## Year2005    -0.85863    0.20203   -4.250    2.14e-05 ***
## Year2006    -1.15129    0.19821   -5.808    6.31e-09 ***
## Year2007    -1.79123    0.20634   -8.681    < 2e-16 ***
## Year2008    -1.39804    0.19812   -7.057    1.70e-12 ***
## Year2009    -0.97778    0.20709   -4.722    2.34e-06 ***
## Year2010    -1.28890    0.21549   -5.981    2.21e-09 ***
## Year2011    -1.23465    0.22456   -5.498    3.84e-08 ***
## Year2012    -1.55176    0.22319   -6.953    3.58e-12 ***
## Year2013    -1.86393    0.25445   -7.325    2.38e-13 ***
## Year2014    -2.22924    0.25144   -8.866    < 2e-16 ***
## Year2015    -2.30106    0.27772   -8.286    < 2e-16 ***
## Year2016    -1.77091    0.25932   -6.829    8.55e-12 ***
## Year2017    -1.89846    0.24875   -7.632    2.31e-14 ***
## Year2018    -1.95854    0.24963   -7.846    4.31e-15 ***
## areaS       -0.54536    0.06031   -9.043    < 2e-16 ***
## season2     -0.11117    0.07719   -1.440    0.14980
## season3     -0.19870    0.08398   -2.366    0.01799  *
## season4     -0.41141    0.07632   -5.391    7.01e-08 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

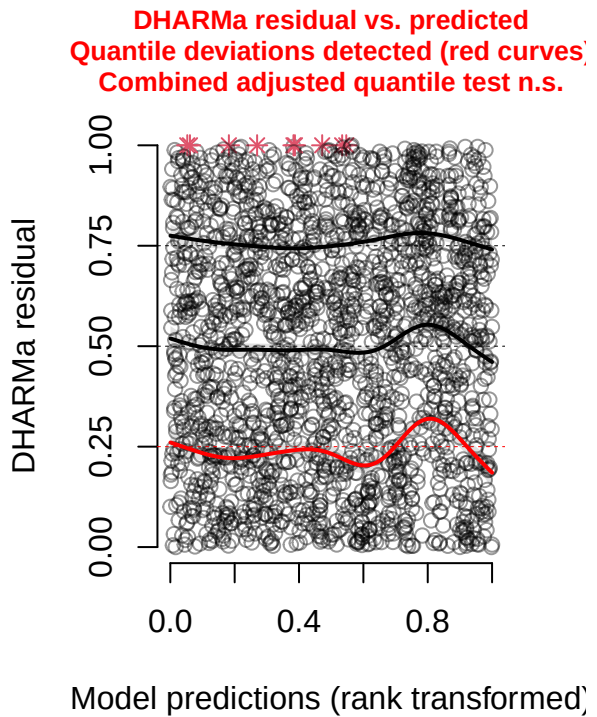
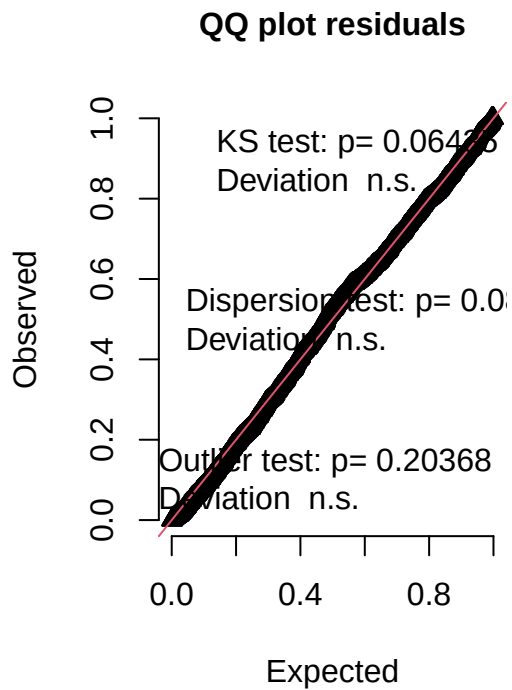
```
plot(simulateResiduals(NB2fit))
```

```

## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully

```

DHARMA residual

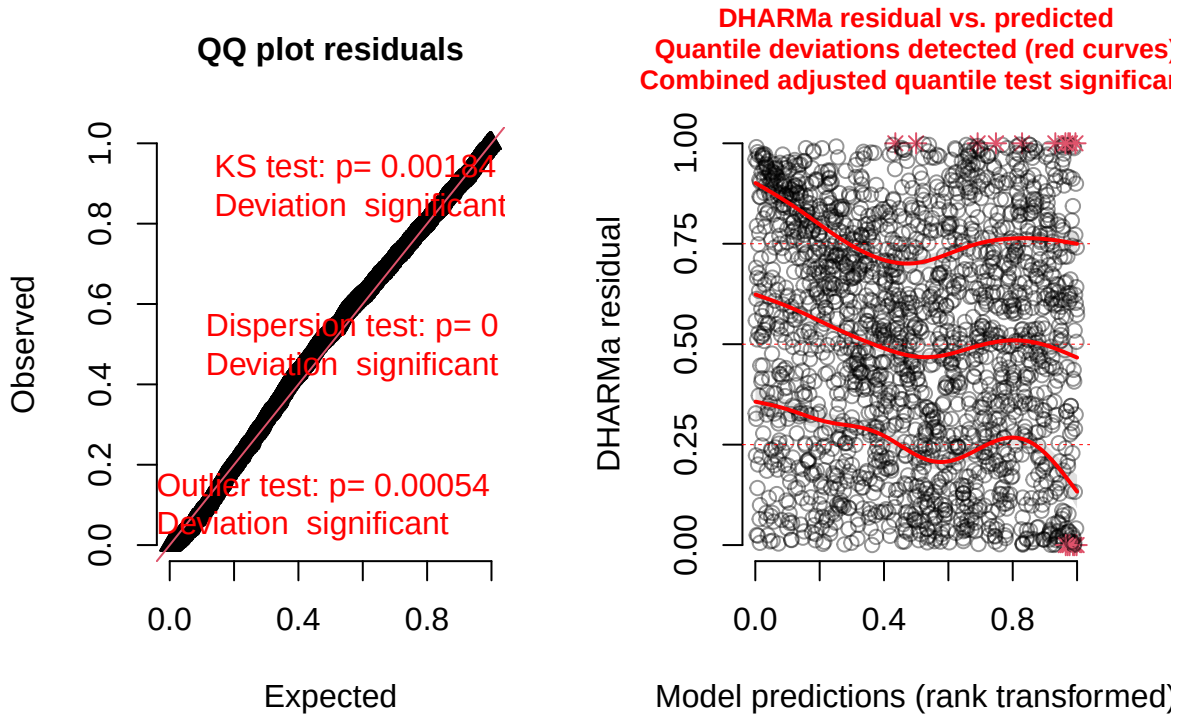


Looks fine.

```
NB1fit <- glmmTMB(BUM ~ Year + area + season + offset(log(sets)),
  data = obsdat,
  family = "nbinom1")
plot(simulateResiduals(NB1fit))
```

DHARMA:testOutliers with type = binomial may have inflated Type I error rates for integer-valued dis

DHARMa residual



stick with NB2 and find best predictors using dredge

```
NB2fit<-glmmTMB(BUM~(Year+area+season)^2+offset(log(sets)),
  data=obsdat,
  family="nbinom2",
  na.action = "na.fail")
#Must set na.action to "na.fail"

NB2dredge<-dredge(NB2fit,rank = "BIC")
head(NB2dredge)
```

```
## Global model call: glmmTMB(formula = BUM ~ (Year + area + season)^2 + offset(log(sets)),
##   data = obsdat, family = "nbinom2", na.action = "na.fail",
##   ziformula = ~0, dispformula = ~1)
## ---
## Model selection table
##   cnd((Int)) dsp((Int)) cnd(are) cnd(ssn) cnd(Yer) cnd(are:ssn) cnd(are:Yer)
## 80   -0.23960          +        +        +        +              +
## 72   -0.06447          +        +        +        +
## 70   -0.18540          +        +        +        +
## 96   -1.04000          +        +        +        +              +
## 71   -0.43240          +        +        +        +
## 69   -0.57170          +        +        +        +
##   cnd(off(log(sts))) df    logLik    BIC  delta weight
## 80                   + 37 -5074.405 10429.7  0.00      1
## 72                   + 34 -5102.233 10462.6 32.88      0
```

```
## 70          + 31 -5117.090 10469.5  39.82      0
## 96          + 65 -5010.002 10513.5  83.78      0
## 71          + 33 -5142.425 10535.4 105.67      0
## 69          + 30 -5158.518 10544.8 115.08      0
## Models ranked by BIC(x)
```

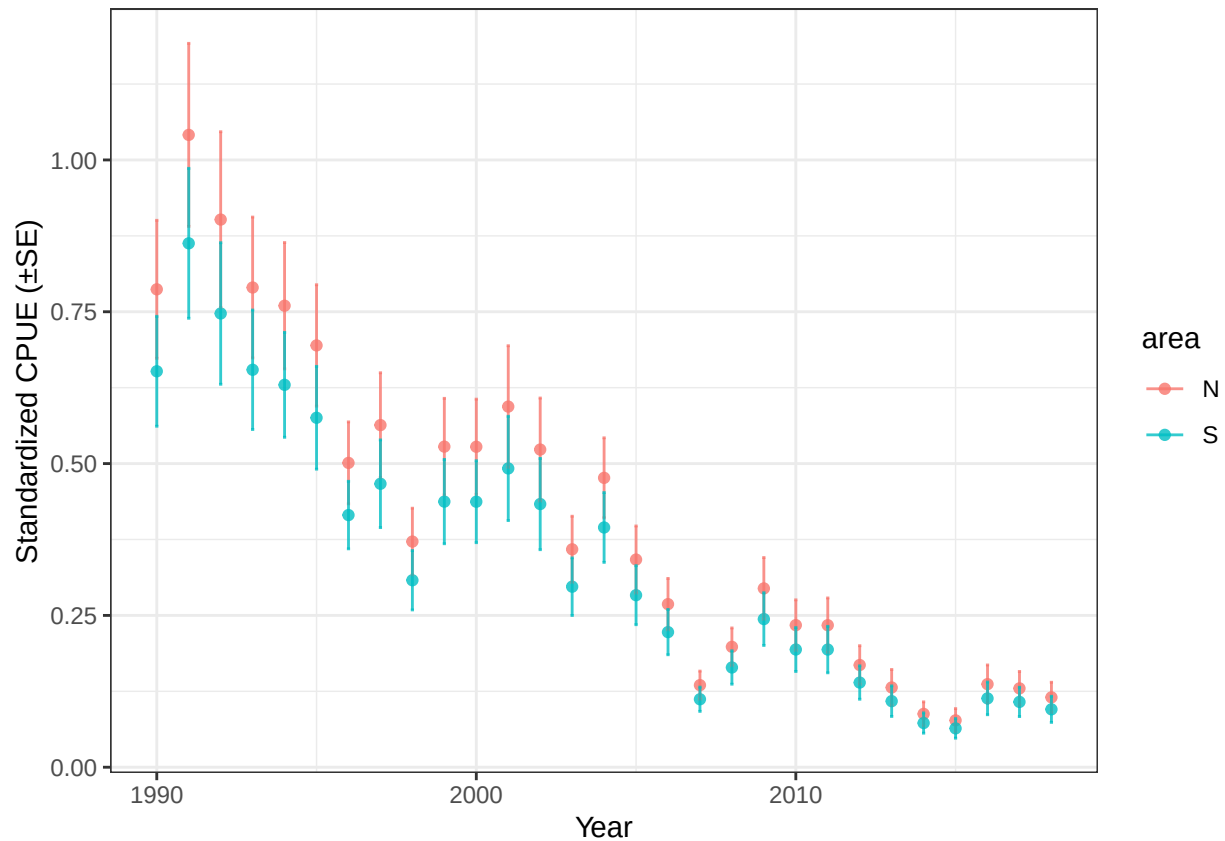
```
NB2best<-get.models(NB2dredge,subset=1)[[1]]
#summary(NB2best)
```

Best model includes all main effects, plus interactions (but not with year)

Prediction, for N and S separately

```
newdata<-expand.grid(Year=factor(levels(obsdat$Year)),
                     area=factor(levels(obsdat$area)),
                     season=factor("1"),
                     sets=1) #1 set to get CPUE
temp<-predict(NB2best,
              newdata = newdata,
              se.fit = TRUE,
              type="response")
newdata$fit<-temp$fit
newdata$se<-temp$se.fit
ggplot(newdata,aes(x=as.numeric(as.character(Year)),y=fit,ymin=fit-se,ymax=fit+se,color=area))+
  geom_point(alpha=0.8)+
  geom_errorbar(width=0.1,alpha=0.8)+
  labs(x="Year",
       y="Standardized CPUE ( $\pm$ SE)",
       main="Negative binomial fit")
```

```
## Ignoring unknown labels:
## * main : "Negative binomial fit"
```



Tweedie

```
TweedieFit<-glmmTMB(BUM/sets~(Year+area+season)^2,
  data=obsdat,
  family="tweedie",
  na.action = "na.fail")
#Must set na.action to "na.fail"

TweedieDredge<-dredge(TweedieFit,rank = "BIC")
```

Fixed terms are "cond((Int))" and "disp((Int))"

```
head(TweedieDredge)
```

```
## Global model call: glmmTMB(formula = BUM/sets ~ (Year + area + season)^2, data = obsdat,
##   family = "tweedie", na.action = "na.fail", ziformula = ~0,
##   dispformula = ~1)
## ---
## Model selection table
##   cnd((Int)) dsp((Int)) cnd(are) cnd(ssn) cnd(Yer) cnd(are:ssn) cnd(are:Yer)
## 16   -0.4306         +         +         +         +         +
##  8   -0.2840         +         +         +         +
##  6   -0.3796         +         +         +
## 32   -1.0570         +         +         +         +         +
##  7   -0.5032         +         +         +
```

```
## 5      -0.6092      +      +
##      df      logLik      BIC      delta weight
## 16 38 -1359.537 3007.6    0.00    1
## 8  35 -1398.003 3061.7   54.16    0
## 6  32 -1411.566 3066.1   58.50    0
## 32 66 -1289.798 3080.7   73.11    0
## 7  34 -1427.210 3112.6  104.98    0
## 5  31 -1441.730 3118.8  111.24    0
## Models ranked by BIC(x)
```

```
TweedieBest<-get.models(TweedieDredge,subset=1)[[1]]
summary(TweedieBest)
```

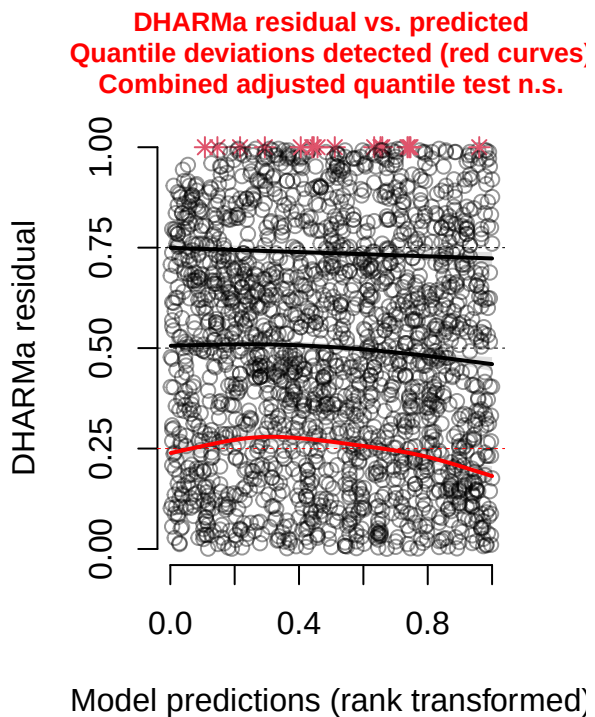
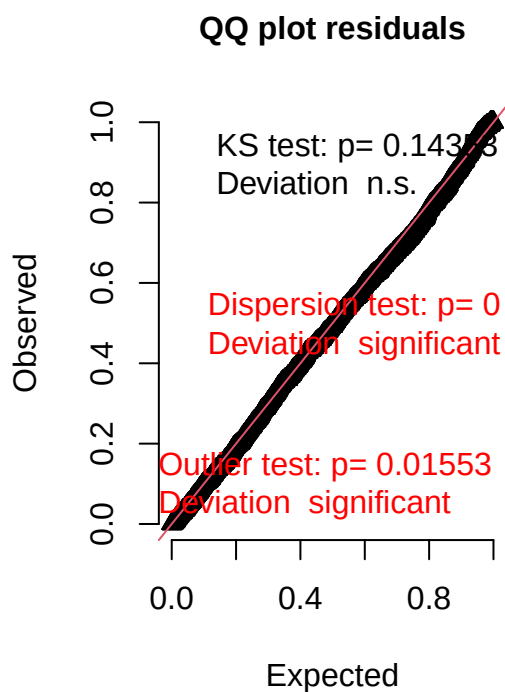
```
## Family: tweedie ( log )
## Formula:      BUM/sets ~ area + season + Year + area:season + 1
## Data: obsdat
##
##      AIC      BIC      logLik -2*log(L)  df.resid
##    2795.1    3007.6   -1359.5    2719.1     1945
##
##
## Dispersion parameter for tweedie family (): 0.717
##
## Conditional model:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.43057    0.12006  -3.586 0.000335 ***
## areaS        -0.07950    0.09231  -0.861 0.389076
## season2      -0.06100    0.09589  -0.636 0.524671
## season3       0.41312    0.09681   4.267 1.98e-05 ***
## season4      -0.21286    0.09599  -2.218 0.026585 *
## Year1991      0.35301    0.14448   2.443 0.014551 *
## Year1992      0.18800    0.15697   1.198 0.231047
## Year1993      0.02322    0.15537   0.149 0.881206
## Year1994     -0.01792    0.14635  -0.122 0.902533
## Year1995     -0.06106    0.15402  -0.396 0.691765
## Year1996     -0.35002    0.15243  -2.296 0.021663 *
## Year1997     -0.23571    0.16325  -1.444 0.148764
## Year1998     -0.62629    0.17204  -3.640 0.000272 ***
## Year1999     -0.17707    0.15978  -1.108 0.267770
## Year2000     -0.17646    0.15844  -1.114 0.265412
## Year2001     -0.08005    0.17077  -0.469 0.639229
## Year2002     -0.22993    0.17195  -1.337 0.181154
## Year2003     -0.62643    0.17601  -3.559 0.000372 ***
## Year2004     -0.34141    0.15863  -2.152 0.031372 *
## Year2005     -0.72350    0.18508  -3.909 9.27e-05 ***
## Year2006     -0.89116    0.18393  -4.845 1.26e-06 ***
## Year2007     -1.70715    0.22146  -7.709 1.27e-14 ***
## Year2008     -1.22189    0.19202  -6.363 1.98e-10 ***
## Year2009     -0.77496    0.19210  -4.034 5.48e-05 ***
## Year2010     -1.09464    0.21267  -5.147 2.65e-07 ***
## Year2011     -0.93975    0.21288  -4.414 1.01e-05 ***
## Year2012     -1.42182    0.23069  -6.163 7.13e-10 ***
## Year2013     -1.65275    0.26974  -6.127 8.95e-10 ***
## Year2014     -2.14579    0.30621  -7.007 2.43e-12 ***
```

```
## Year2015      -2.31256    0.33591   -6.884 5.80e-12 ***
## Year2016      -1.51939    0.27154   -5.595 2.20e-08 ***
## Year2017      -1.73014    0.27459   -6.301 2.96e-10 ***
## Year2018      -1.79363    0.26551   -6.756 1.42e-11 ***
## areaS:season2 -0.08381    0.13972   -0.600 0.548631
## areaS:season3 -1.31206    0.15722   -8.346 < 2e-16 ***
## areaS:season4 -0.34278    0.14306   -2.396 0.016570 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
plot(simulateResiduals(TweedieBest))
```

```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

DHARMa residual



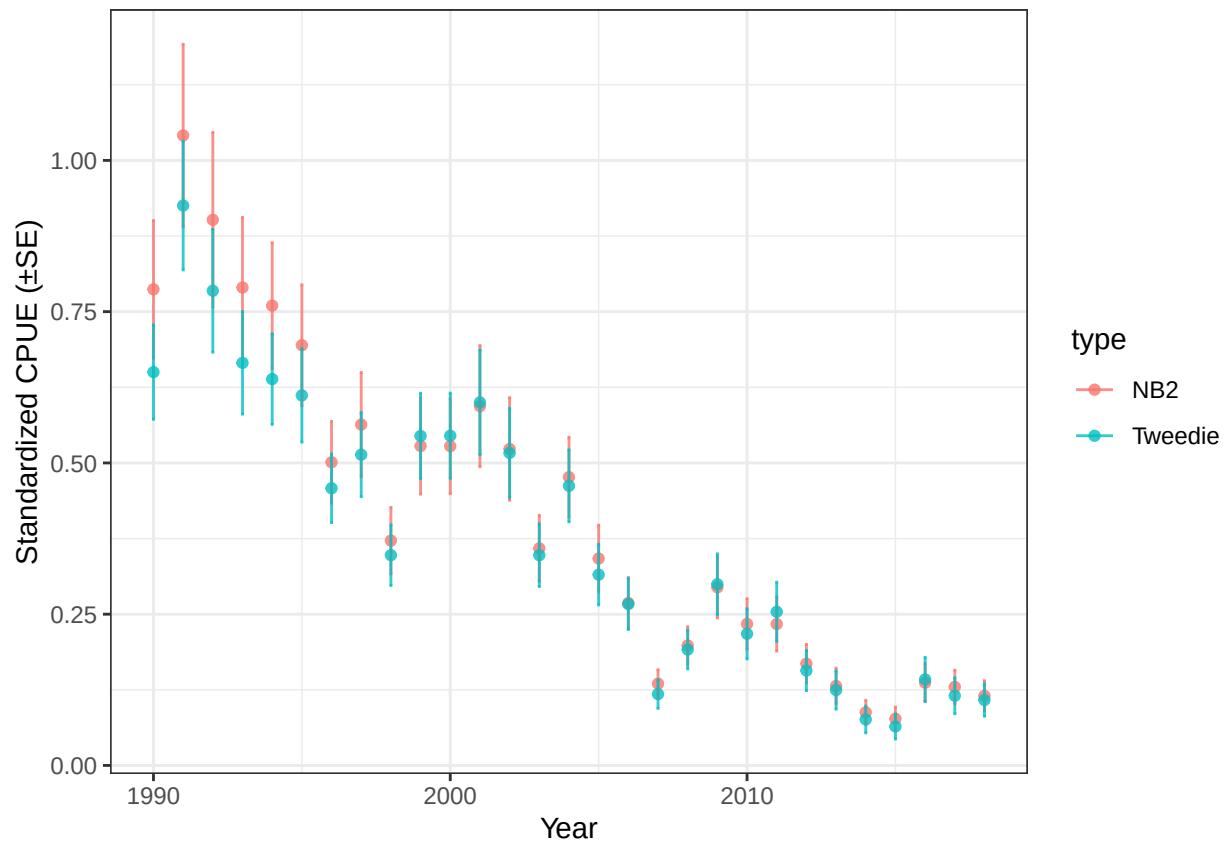
Residuals look fine. Prediction

```
newdata2<-expand.grid(Year=factor(levels(obsdat$Year)),
                      area=factor("N"),
                      season=factor("1"),
                      sets=1) #1 set to get CPUE
temp<-predict(TweedieBest,
              newdata = newdata2,
              se.fit = TRUE,
              type="response")
```

```

newdata2$fit<-temp$fit
newdata2$se<-temp$se.fit
bind_rows(list(NB2=filter(newdata,area=="N"),
  Tweedie=newdata2),.id="type") %>%
ggplot(aes(x=as.numeric(as.character(Year)),y=fit,ymin=fit-se,
  ymax=fit+se,color=type))+
  geom_point(alpha=0.8)+
  geom_errorbar(width=0.1,alpha=0.8)+
  labs(x="Year",
    y="Standardized CPUE (±SE)")

```



Delta-lognormal model

Will need the lognormal conversion functions. These calculate the mean and standard error of a lognormal function from the mean and standard error of the normally distributed variable on the log scale. Also need the function for Lo et al. to calculate the standard error of the index using the delta method.

```

lnorm.mean=function(x1,x1e) {
  exp(x1+0.5*x1e^2)
}

lnorm.se=function(x1,x1e) {
  ((exp(x1e^2)-1)*exp(2*x1+x1e^2))^0.5
}

lo.se=function(x1,x1e,x2,x2e) {
  if(length(x1[!is.na(x1) &!is.na(x2)])>1 & length(unique(x1))>1 & length(unique(x2))>1 )
    cor12=cor(x1[!is.na(x1) &!is.na(x2)],x2[!is.na(x1) &!is.na(x2)]) else cor12=0
}

```

```
(x1e^2 * x2^2 + x2e^2*x1^2+2*x1*x2*cor12*x1e*x2e)^0.5
}
```

Part 1. binomial

```
obsdat$present<-ifelse(obsdat$BUM>0,1,0)
BinomFit<-glmmTMB(present~Year+area*season,
                  data=obsdat,
                  family="binomial",
                  na.action = "na.fail")
BinDredge<-dredge(BinomFit,rank = "BIC")
```

```
## Fixed terms are "cond((Int))" and "disp((Int))"
```

```
head(BinDredge)
```

```
## Global model call: glmmTMB(formula = present ~ Year + area * season, data = obsdat,
##   family = "binomial", na.action = "na.fail", ziformula = ~0,
##   dispformula = ~1)
## ---
## Model selection table
##      cnd((Int)) dsp((Int)) cnd(are) cnd(ssn) cnd(Ver) cnd(are:ssn) df    logLik
## 12      1.802          +          +          +          +      8 -1088.711
##  4      2.150          +          +          +          +      5 -1117.745
##  3      1.825          +          +          +          +      4 -1139.531
## 16      1.781          +          +          +          +     36 -1033.070
##  2      1.315          +          +          +          +      2 -1163.533
##  8      2.030          +          +          +          +     33 -1060.412
##      BIC  delta weight
## 12 2238.2    0.00      1
##  4 2273.5   35.29      0
##  3 2309.4   71.27      0
## 16 2339.5  101.30      0
##  2 2342.3  104.09      0
##  8 2371.4  133.21      0
## Models ranked by BIC(x)
```

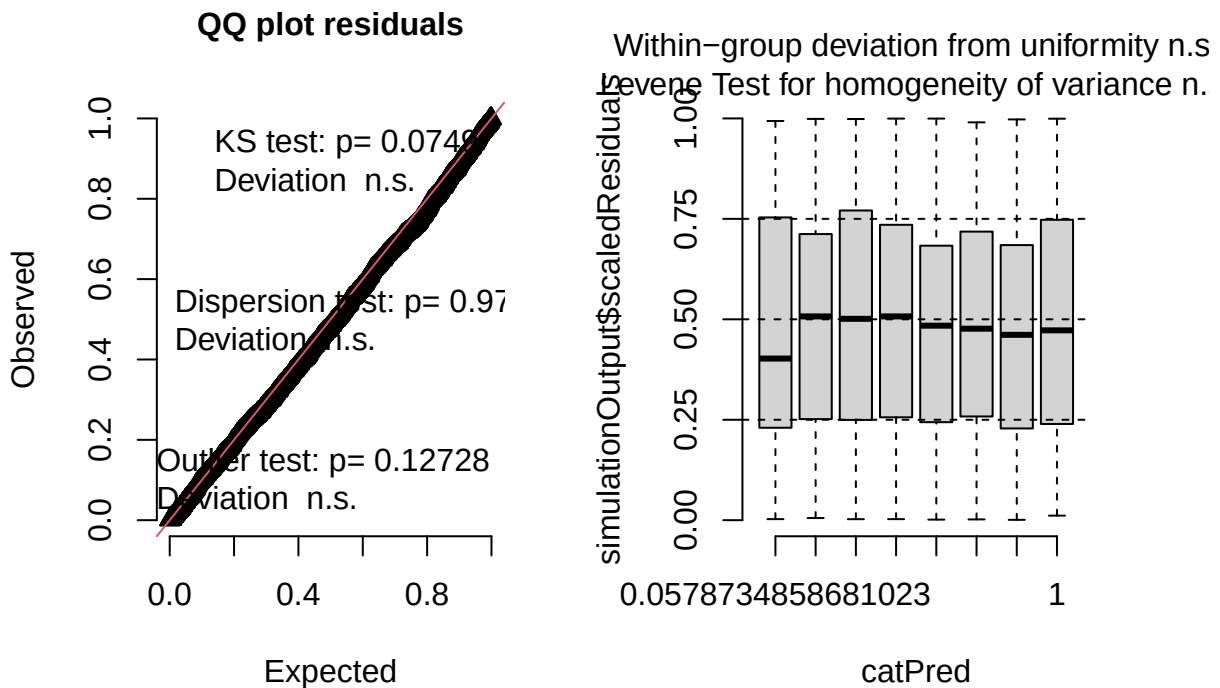
```
BinBest<-get.models(BinDredge,subset=1)[[1]]
summary(BinBest)
```

```
## Family: binomial ( logit )
## Formula:          present ~ area + season + area:season + 1
## Data: obsdat
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    2193.4    2238.2   -1088.7    2177.4    1975
##
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    1.80171    0.15264   11.803 < 2e-16 ***
```

```
## areaS      0.05643    0.23937    0.236    0.81364
## season2   -0.60546    0.21987   -2.754    0.00589 **
## season3    0.05323    0.27126    0.196    0.84443
## season4   -1.19011    0.20466   -5.815    6.06e-09 ***
## areaS:season2 -1.00440    0.31389   -3.200    0.00138 **
## areaS:season3 -2.15556    0.35574   -6.059    1.37e-09 ***
## areaS:season4 -0.07914    0.30212   -0.262    0.79336
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
plot(simulateResiduals(BinBest))
```

DHARMA residual



Part 2. log(CPUE) positive only

```
posdat<-filter(obsdat,present==1)%>%
  mutate(cpue=BUM/sets)
cpueFit<-glmmTMB(log(cpue)~Year+area*season,
  data=posdat,
  family="gaussian",
  na.action = "na.fail")
cpueDredge<-dredge(cpueFit,rank = "BIC")
```

```
## Fixed terms are "cond((Int))" and "disp((Int))"
```



```
head(cpueDredge)
```

```
## Global model call: glmmTMB(formula = log(cpue) ~ Year + area * season, data = posdat,
##   family = "gaussian", na.action = "na.fail", ziformula = ~0,
##   dispformula = ~1)
## ---
## Model selection table
##      cnd((Int)) dsp((Int)) cnd(are) cnd(ssn) cnd(Ver) cnd(are:ssn) df      logLik
## 16      -0.6505          +          +          +          +          + 37 -1932.094
## 8       -0.5858          +          +          +          +          + 34 -1948.791
## 7       -0.6929          +          +          +          +          + 33 -1956.178
## 6       -0.6583          +          +          +          +          + 31 -1965.631
## 5       -0.7845          +          +          +          +          + 30 -1974.845
## 12      -1.2250          +          +          +          +          + 9  -2094.101
##      BIC  delta weight
## 16 4132.5   0.00  0.997
## 8  4144.1  11.64  0.003
## 7  4151.7  19.16  0.000
## 6  4156.1  23.57  0.000
## 5  4167.2  34.74  0.000
## 12 4253.5 120.97  0.000
## Models ranked by BIC(x)
```

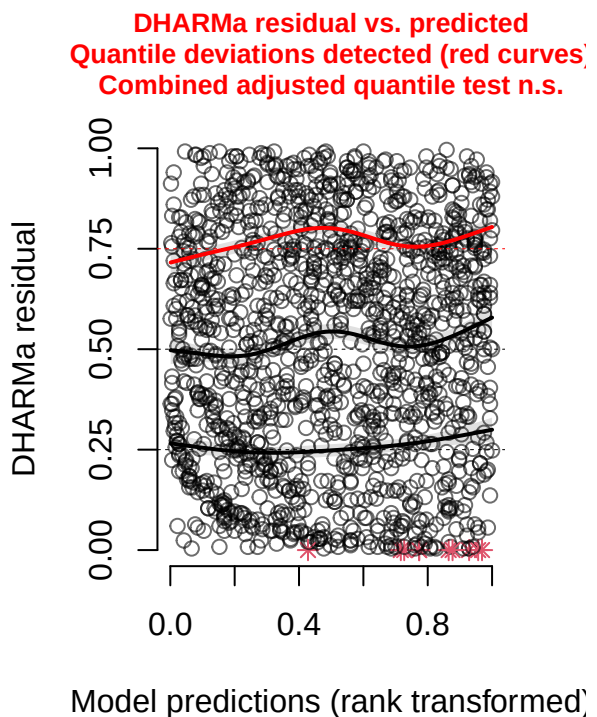
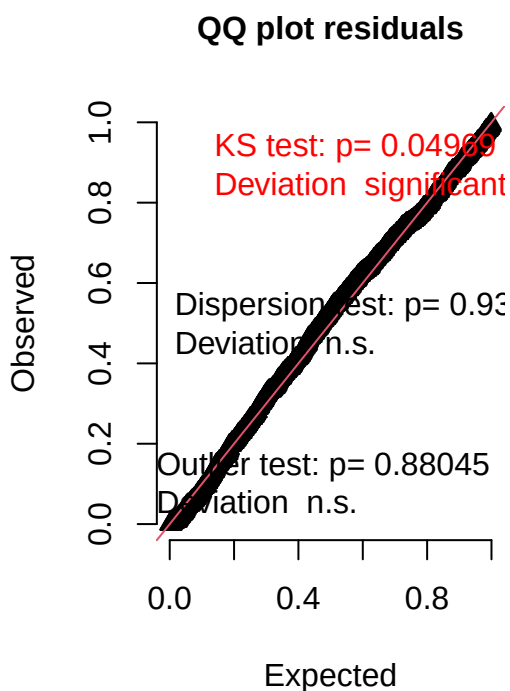
```
cpueBest<-get.models(cpueDredge,subset=1)[[1]]
summary(cpueBest)
```

```
## Family: gaussian ( identity )
## Formula:      log(cpue) ~ area + season + Year + area:season + 1
## Data: posdat
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    3938.2    4132.5   -1932.1    3864.2     1373
##
##
## Dispersion estimate for gaussian family (sigma^2): 0.907
##
## Conditional model:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept)  -0.65050    0.12961  -5.019 5.20e-07 ***
## areaS        -0.04753    0.08691  -0.547  0.58448
## season2      -0.05960    0.09205  -0.647  0.51731
## season3       0.18998    0.09662   1.966  0.04926 *
## season4      -0.21381    0.09621  -2.222  0.02626 *
## Year1991       0.35200    0.16945   2.077  0.03777 *
## Year1992       0.21551    0.17732   1.215  0.22421
## Year1993      -0.06036    0.16757  -0.360  0.71869
## Year1994       0.06654    0.16193   0.411  0.68113
## Year1995      -0.03463    0.16586  -0.209  0.83459
## Year1996      -0.21564    0.16122  -1.338  0.18105
## Year1997      -0.40289    0.16625  -2.423  0.01538 *
## Year1998      -0.43965    0.17768  -2.474  0.01334 *
## Year1999      -0.30180    0.17333  -1.741  0.08165 .
```

```
## Year2000      -0.19329    0.17236   -1.121    0.26209
## Year2001      -0.12511    0.18127   -0.690    0.49008
## Year2002      -0.20176    0.17880   -1.128    0.25915
## Year2003      -0.59509    0.17674   -3.367    0.00076 ***
## Year2004      -0.43353    0.16462   -2.634    0.00845 **
## Year2005      -0.57345    0.18476   -3.104    0.00191 **
## Year2006      -0.81570    0.18050   -4.519    6.21e-06 ***
## Year2007      -1.39872    0.18885   -7.406    1.30e-13 ***
## Year2008      -1.05625    0.17961   -5.881    4.09e-09 ***
## Year2009      -0.81103    0.18233   -4.448    8.66e-06 ***
## Year2010      -0.70946    0.20443   -3.470    0.00052 ***
## Year2011      -0.87601    0.19704   -4.446    8.76e-06 ***
## Year2012      -1.21964    0.19345   -6.305    2.89e-10 ***
## Year2013      -1.07183    0.24438   -4.386    1.16e-05 ***
## Year2014      -1.32646    0.25492   -5.204    1.96e-07 ***
## Year2015      -1.44784    0.27402   -5.284    1.27e-07 ***
## Year2016      -1.25887    0.22893   -5.499    3.82e-08 ***
## Year2017      -1.34998    0.24526   -5.504    3.71e-08 ***
## Year2018      -1.46261    0.20916   -6.993    2.69e-12 ***
## areaS:season2  0.17326    0.13929    1.244    0.21352
## areaS:season3 -0.72471    0.15405   -4.704    2.55e-06 ***
## areaS:season4 -0.29605    0.13779   -2.149    0.03167 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
plot(simulateResiduals(cpueBest))
```

DHARMA residual

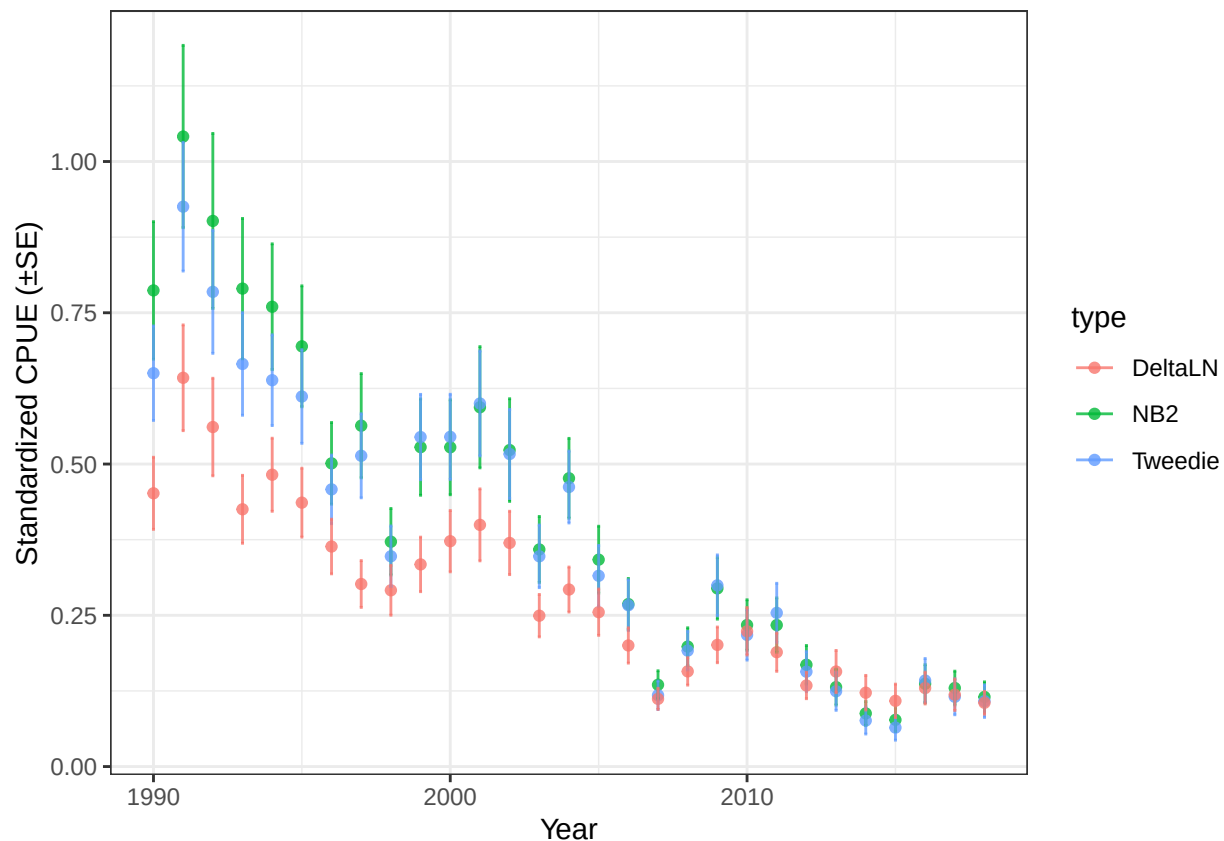


Calculate index

```
newdata3<-expand.grid(Year=factor(levels(obsdat$Year)),
                      area=factor("N"),
                      season=factor("1"),
                      sets=1) #1 set to get CPUE
temp<-predict(BinBest,
              newdata = newdata3,
              se.fit = TRUE,
              type="response")
newdata3$binomial<-temp$fit
newdata3$binomial.se<-temp$se.fit
temp<-predict(cpueBest,
              newdata = newdata3,
              se.fit = TRUE) #response is on log scale
#lognormal conversion functions to get mean if positive
newdata3$pos.cpue<-lnorm.mean(temp$fit,temp$se.fit)
newdata3$pos.cpue.se<-lnorm.se(temp$fit,temp$se.fit)
#index
newdata3<-newdata3 %>%
  mutate(fit=binomial*pos.cpue,
         se=lo.se(binomial,binomial.se,pos.cpue,pos.cpue.se))
```

Compare fits

```
bind_rows(list(NB2=filter(newdata,area=="N"),
               Tweedie=newdata2,
               DeltaLN=newdata3),.id="type") %>%
ggplot(aes(x=as.numeric(as.character(Year)),y=fit,ymin=fit-se,
           ymax=fit+se,color=type))+
  geom_point(alpha=0.8)+
  geom_errorbar(width=0.1,alpha=0.8)+
  labs(x="Year",
       y="Standardized CPUE ( $\pm$ SE)")
```



Lognormal is a little flatter.

Random effect

```
NB2Random<-glmmTMB(BUM~Year+area+season+(1|area:Year)+offset(log(sets)),
  data=obsdat,
  family="nbinom2")

summary(NB2Random)
```

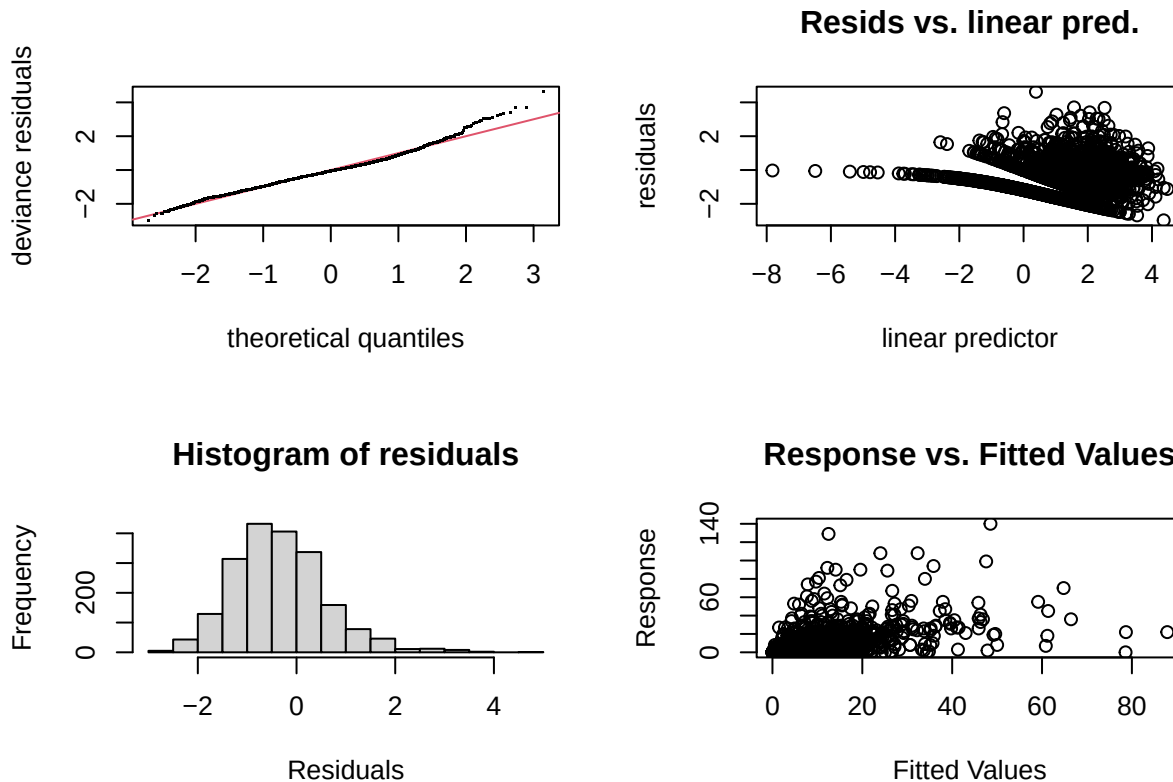
```
## Family: nbinom2 ( log )
## Formula:
## BUM ~ Year + area + season + (1 | area:Year) + offset(log(sets))
## Data: obsdat
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##  10258.8  10454.5   -5094.4   10188.8     1948
##
## Random effects:
##
## Conditional model:
## Groups   Name      Variance Std.Dev.
## area:Year (Intercept) 0.04718  0.2172
## Number of obs: 1983, groups: area:Year, 58
##
## Dispersion parameter for nbinom2 family (): 0.84
##
```

```
## Conditional model:
##      Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.16097    0.20931  -0.769 0.441860
## Year1991     0.34192    0.28563   1.197 0.231266
## Year1992     0.21422    0.29413   0.728 0.466415
## Year1993     0.02933    0.28554   0.103 0.918193
## Year1994    -0.01653    0.28111  -0.059 0.953120
## Year1995    -0.03302    0.28361  -0.116 0.907326
## Year1996    -0.30421    0.27875  -1.091 0.275131
## Year1997    -0.24889    0.28841  -0.863 0.388162
## Year1998    -0.72282    0.28838  -2.506 0.012195 *
## Year1999    -0.28799    0.28930  -0.995 0.319512
## Year2000    -0.29462    0.28723  -1.026 0.305007
## Year2001    -0.19204    0.29947  -0.641 0.521348
## Year2002    -0.41229    0.30032  -1.373 0.169798
## Year2003    -0.74679    0.29042  -2.571 0.010128 *
## Year2004    -0.43280    0.28451  -1.521 0.128209
## Year2005    -0.71812    0.30388  -2.363 0.018121 *
## Year2006    -1.03562    0.29378  -3.525 0.000423 ***
## Year2007    -1.68264    0.29941  -5.620 1.91e-08 ***
## Year2008    -1.31710    0.29468  -4.470 7.83e-06 ***
## Year2009    -0.86229    0.29999  -2.874 0.004048 **
## Year2010    -1.20241    0.30649  -3.923 8.74e-05 ***
## Year2011    -1.12031    0.31235  -3.587 0.000335 ***
## Year2012    -1.44694    0.31135  -4.647 3.36e-06 ***
## Year2013    -1.79630    0.33688  -5.332 9.71e-08 ***
## Year2014    -2.15254    0.33486  -6.428 1.29e-10 ***
## Year2015    -2.18199    0.35234  -6.193 5.91e-10 ***
## Year2016    -1.67976    0.33905  -4.954 7.26e-07 ***
## Year2017    -1.89652    0.33635  -5.639 1.71e-08 ***
## Year2018    -1.87064    0.33273  -5.622 1.89e-08 ***
## areaS        -0.59236    0.08406  -7.047 1.83e-12 ***
## season2      -0.10823    0.07678  -1.410 0.158642
## season3      -0.16247    0.08386  -1.937 0.052707 .
## season4      -0.38568    0.07627  -5.056 4.27e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Gam

GAMs can be fit with the mgcv library

```
NB2Gam<-gam(BUM~Year+season*area+te(lon,lat),data=obsdat,
            family="nb") #Negative binomial 2
gam.check(NB2Gam)
```



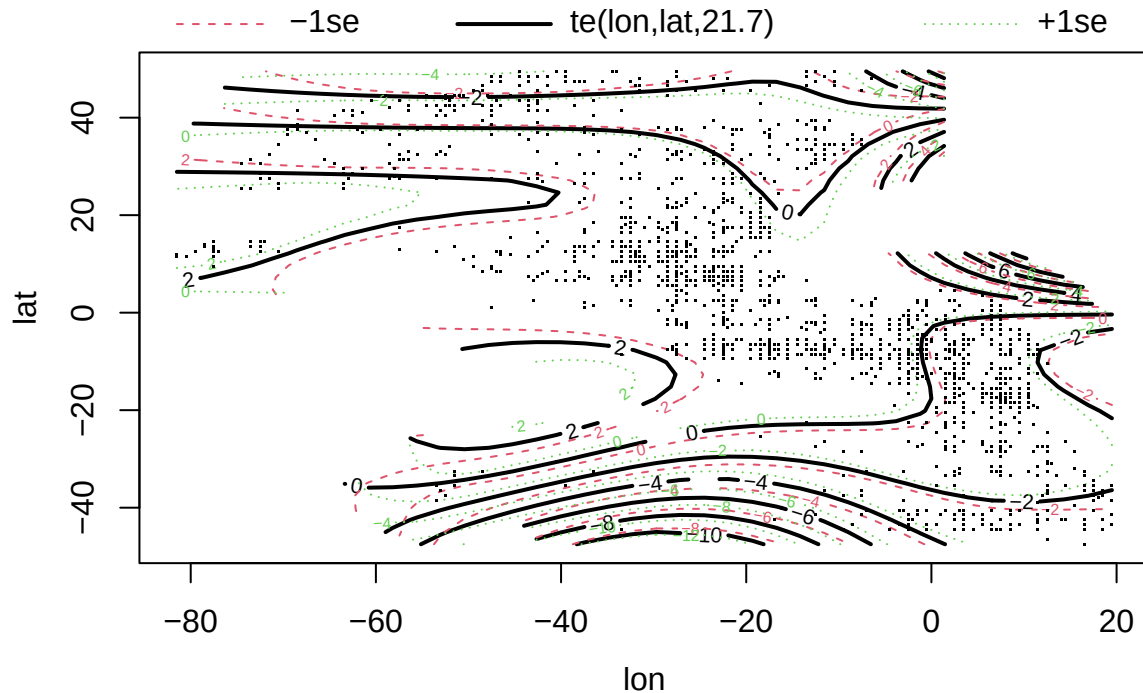
```
##
## Method: REML   Optimizer: outer newton
## full convergence after 5 iterations.
## Gradient range [-0.0003240759,0.002123064]
## (score 4984.132 & scale 1).
## Hessian positive definite, eigenvalue range [1.81093,510.9759].
## Model rank = 60 / 60
##
## Basis dimension (k) checking results. Low p-value (k-index<1) may
## indicate that k is too low, especially if edf is close to k'.
##
##           k'   edf k-index p-value
## te(lon,lat) 24.0 21.7   0.76 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(NB2Gam)
```

```
##
## Family: Negative Binomial(1.013)
## Link function: log
##
## Formula:
## BUM ~ Year + season * area + te(lon, lat)
##
```

```
## Parametric coefficients:
##           Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.4693560  0.1635277  8.985 < 2e-16 ***
## Year1991     -0.1382851  0.1726216 -0.801  0.4231
## Year1992      0.1039777  0.1813552  0.573  0.5664
## Year1993      0.3268669  0.1706019  1.916  0.0554 .
## Year1994      0.2611382  0.1628006  1.604  0.1087
## Year1995      0.2017464  0.1693802  1.191  0.2336
## Year1996      0.4442852  0.1653321  2.687  0.0072 **
## Year1997      0.1476247  0.1754829  0.841  0.4002
## Year1998     -0.2313013  0.1814364 -1.275  0.2024
## Year1999     -0.2499760  0.1825539 -1.369  0.1709
## Year2000     -0.0495849  0.1777759 -0.279  0.7803
## Year2001      0.0376599  0.1899353  0.198  0.8428
## Year2002     -0.0003795  0.1865505 -0.002  0.9984
## Year2003     -0.2127646  0.1848828 -1.151  0.2498
## Year2004     -0.0692976  0.1729257 -0.401  0.6886
## Year2005     -0.1880593  0.1894300 -0.993  0.3208
## Year2006     -0.7710197  0.1901587 -4.055 5.02e-05 ***
## Year2007     -0.9599433  0.2028102 -4.733 2.21e-06 ***
## Year2008     -0.8937982  0.1900449 -4.703 2.56e-06 ***
## Year2009     -1.1751932  0.1962713 -5.988 2.13e-09 ***
## Year2010     -1.0317030  0.2067115 -4.991 6.01e-07 ***
## Year2011     -0.9178000  0.2121632 -4.326 1.52e-05 ***
## Year2012     -0.8328623  0.2126395 -3.917 8.97e-05 ***
## Year2013     -1.0868502  0.2448492 -4.439 9.04e-06 ***
## Year2014     -1.4156021  0.2539803 -5.574 2.49e-08 ***
## Year2015     -1.3586694  0.2767875 -4.909 9.17e-07 ***
## Year2016     -1.5658720  0.2517876 -6.219 5.00e-10 ***
## Year2017     -1.5970035  0.2540105 -6.287 3.23e-10 ***
## Year2018     -1.3602237  0.2336687 -5.821 5.84e-09 ***
## season2      -0.0881568  0.1020246 -0.864  0.3875
## season3       0.0285529  0.1077983  0.265  0.7911
## season4       0.0086388  0.1063402  0.081  0.9353
## areaS        0.1228119  0.2136788  0.575  0.5655
## season2:areaS  0.6688670  0.1580912  4.231 2.33e-05 ***
## season3:areaS -0.2252772  0.1725983 -1.305  0.1918
## season4:areaS -0.1011986  0.1555650 -0.651  0.5154
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##           edf Ref.df Chi.sq p-value
## te(lon,lat) 21.7  22.66  1254 <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.262   Deviance explained =  52%
## -REML = 4984.1   Scale est. = 1           n = 1983
```

```
plot(NB2Gam)
```



Bycatch estimation

BycatchEstimator tool

<https://ebabcock.github.io/BycatchEstimator/>

```
library(BycatchEstimator)
library(MuMIn)
setupObj<-bycatchSetup(
  obsdat = droplevels(LLSIM_BUM_Example_observer[LLSIM_BUM_Example_observer$Year>2010 & LLSIM_BUM_Example_observer$Area=="SWO"],
  logdat = droplevels(LLSIM_BUM_Example_logbook[LLSIM_BUM_Example_logbook$Year>2010 & LLSIM_BUM_Example_logbook$Area=="SWO"],
  yearVar = "Year",
  obsEffort = "hooks",
  logEffort = "hooks",
  obsCatch = c("SWO", "BUM")[2], # selecting Blue marlin catch
  catchUnit = "number",
  catchType = "catch",
  logNum = NA,
  sampleUnit = "trips",
  factorVariables = c("Year", "area", "season"),
  numericVariables = NA,
  EstimateBycatch = TRUE,
  baseDir = getwd(),
  runName = "LLSIMBUMtripExample",
  runDescription = "LLSIMBUM by trip",
```



```

common = c("Swordfish","Blue marlin")[2], # selecting Blue marlin
sp = c("Xiphias gladius","Makaira nigricans")[2], # selecting Blue marlin
reportType = "html"
)
designObj <- bycatchDesign(
  setupObj = setupObj,
  designScenario = "noPool",
  designMethods = c("Ratio", "Delta"),
  designVars = c("Year","area","season"),
  designPooling = FALSE,
  poolTypes=c("adjacent","all","none"),
  pooledVar=c(NA,NA),
  adjacentNum=c(1,NA),
  minStrataUnit = 1,
  reportType="html"
)
bycatchFit(
  setupObj = setupObj,
  modelScenario = "s1",
  complexModel = formula(y~Year+area),
  simpleModel = formula(y~Year),
  indexModel = formula(y~Year),
  modelTry = c("TMBnbinom1","TMBtweedie"),
  randomEffects = NULL,
  randomEffects2 = NULL,
  selectCriteria = "BIC",
  DoCrossValidation = TRUE,
  Cival = 0.05,
  VarCalc = "Simulate",
  useParallel = TRUE,
  nSims = 100,
  plotValidation = FALSE,
  trueVals = NULL,
  trueCols = NULL,
  reportType = "html"
)

```

```
## Fixed terms are "cond(Year)", "cond(offset(log(Effort)))", "cond((Int))" and "disp((Int))"
```

```
## Fixed terms are "cond(Year)", "cond((Int))" and "disp((Int))"
```

```
## [1] "1 Blue marlin, catch, complete, 2025-11-04 12:55:02.346358"
```

```

allResults<-loadOutputs(baseDir = getwd(),
  runName= "LLSIMBUMtripExample",
  runDate = Sys.Date(),
  designScenarios = "noPool",
  modelScenarios = "s1"
)
#Plot all together
ggplot(allResults$allYearEstimates,aes(x=Year,y=Total,
  ymin=TotalLCI,ymax=TotalUCI,

```

```

    fill=Source,color=Source)))+
  geom_line()+
  geom_ribbon(alpha=0.4)+
  theme_bw()

```

